Optimization of an hybrid parallel D2Q9 lattice Boltzmann solver for Kármán vortex street

Technique d'Optimisation de la Parallélisation

Bauer Théo **Nolwen Dolléans**

M1 - CHPS

Soumis le : 28 avril 2026

1. Introduction

Ceci est un projet d'optimisation d'un programme parallèle de simulation de mécanique des fluides en language C++ dans un contexte d'études en Calcul Haute Performance.

L'objectif principal est d'analyser les performances du code existant, d'identifier les principaux goulets d'étranglement, puis de proposer et mettre en oeuvre des optimisations visant à améliorer son efficacité à l'aide principalement d'outils de profiling afin de rendre le programme le plus scalable possible.

La parallélisation est réalisé grâce aux librairies MPI et OpenMP, visant à profiter du modèle physique pour adapter la simulation sur des architectures HPC.

2. Analyse du code initial

Le code original se structure comme suit :

1. Une phase d'initialisation.
2. Envoi des cellules fantômes.
3. Calcul de la vitesse des particules fluides et de la densité de ce dernier en tout points.
4. Actualisation de la position du fluide.
5. Envoi des cellules fantômes.
6. ...

Par conséquent, nous pouvons extraire 3 grands axes d'optimisation :

2.1. Communication entre processus MPI et implémentation OpenMP

Dans cette section, nous allons travailler sur plusieurs axes. Le premier sera sur la communication en elle même. En effet, nous allons chercher à privilégier les communications non bloquantes quand c'est possible, s'assurer que le maillage et la répartition des tâches sont optimales en les comparant avec l'état de l'art pour ce genre de problème, comparer s'il est plus judicieux de faire des communications collectives ou de point à point, limiter les synchronisations entre processus pour des calculs plus fluides.

Cette simulation porte parfaitement à une parallélisation multi-thread, réalisé avec une région parallèle de OpenMP.

2.2. Accès mémoire et optimisations de caches

Les calculs étant juste pour cette première version du simulateur, notre objectif dans cette section va être de se concentrer sur la manière dont les calculs sont réalisés. Ainsi, nous chercherons à optimiser la localité mémoire (spatiale et temporelle) ; à s'assurer que l'allocation mémoire et la déclaration de variables soient pertinentes et à vérifier d'éventuels effets NUMA.

2.3. Micro-optimisations de calculs inutiles ou bugs éventuels

En fonction de notre analyse de la structure de certains calculs, il est possible de rencontrer certaines parties qui sont inutiles pour l'obtention du résultat final, tels que des valeurs étant réécrites sans utilisation particulière, des fonctions non-utilisées ou encore une structure de code n'assurant pas l'obtention de bugs dans tout les cas possibles. Ainsi, lorsque nous rencontrons une micro-optimisation réalisable, nous la mettons en place sans toujours l'avoir repérée depuis un profiler.

3. Optimisations apportées

Nous utiliserons plusieurs outils de profiling pour déceler les points chauds du programme :

- *Instruments.app* : Flame Graph, nombre d'instructions et de cycles CPU, IPC (Instructions per cycles) moyens
- *perf* : Mesure des IPC, cache-misses, cache-references par fonction
- *gdb* : Débuggage, évaluation de l'évolution des variables et repérage de crash.

Tout graphique et résultats d'outils de profiling seront mis en annexes.

3.1. Première approche

Afin de commencer à chercher les premières optimisations, nous avons tracer un Flame graph du programme (cf

Annexe A). Nous pouvons ainsi nous rendre compte que les communications MPI pour un maillage en halo sont le point chaud majeur, et ralentissent significativement le programme. C'est donc la première chose sur laquelle nous allons nous concentrer.

3.2. Optimisation des communications MPI

La première de ces optimisations est le fait de retirer le 2e envoi de maille fantôme vers la gauche, puisqu'il n'est pas nécessaire de le faire 2 fois.

Nous constatons que les performances sont réduites à cause de 3 fonctions MPI : *MPI_Barrier*, *MPI_Send* et *MPI_Recv* dans la fonction *lbm_comm_halo_exchange* (cf Annexe B). Nous allons donc repenser comment les mailles fantômes sont envoyées.
Les *MPI_Barrier* sont un problème majeur dans cette fonction. En effet, il est inutile de mettre des MPI_Barrier ici, puisque le buffer envoyé est différent pour chacune des fonctions d'envoi et les communications étant bloquante, la synchronisation est déjà incluse.
Comme dit précédemment, les buffers envoyés par communication sont différents (cf. Figure 1).

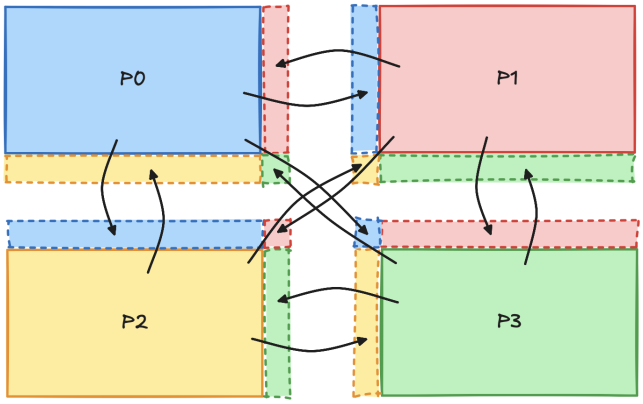


FIGURE 1 – Halo exchange communication scheme for 4 MPI processes

Ainsi, nous pouvons utiliser des communications non-bloquantes sur chacune des fonctions d'envoi de mailles à l'aide de *MPI_Isend* et *MPI_Irecv*, avec un *MPI_Waitall* à la place de la série de *MPI_Barrier*, pour s'assurer que le programme calcul sur une zone qui n'a pas encore été complètement reçue puis avons ajouter un tag unique en fonction du rang et de la direction de la communication afin que les communications soient déterministes. Ceci sera notre première optimisation.

Nous avons également remarquer d'autres *MPI_Barrier* au sein même de la boucle de calcul entre chaque étape, forçant les processus MPI à se synchroniser après le calcul des collisions, de l'échange de mailles fantômes et du calcul de la propagation. Or, cela est sans justification car les processus ont besoin d'être synchronisés uniquement lors de l'envoi de message, pour éviter d'entamer le calcul sur des buffers qui ne sont pas encore pleinement envoyés. Nous pouvons ainsi supprimer l'entièreté des *MPI_Barrier*.

A l'heure actuelle, la division des processus MPI se fait de la manière suivante :

1. nombre de processus en x : par le PGCD de la longueur par le nombre de processus MPI
2. nombre de processus en y : $\frac{Nombre\ de\ processus}{nombre\ de\ processus\ en\ x}$

Cette répartition naïve fonctionne, mais réparti les processus en chaîne et non en halo. En effet, PGCD de comm_size et width vaut comm_size dès que comm_size divise width, on se retrouve donc avec une répartition de (comm_size x 1). De plus, cette méthode ne prend pas en compte la taille de la conduite : le PGCD sera toujours égal au nombre de processus en x, même si la largeur est plus grande que la longueur.
Une modification que l'on peut apporter est de trouver un nb_x et un nb_y tels que :

$$\begin{cases} \frac{width}{nb_x} = \frac{height}{nb_y} \\ nb_x * nb_y = comm_size \end{cases}$$

$$\Longrightarrow \begin{cases} nb_x = \sqrt{comm_size \frac{width}{height}} \\ nb_y = comm_size / nb_x \end{cases}$$

Cependant, comme nb_x et nb_y sont des entiers, afin de limiter des erreurs suite à des conversions de flottants, nous avons choisi de faire cette algorithme : on détermine la paire $(i \leq comm_size, j = comm_size/i)$ minimisant la fonction suivante $\left| j - \sqrt{comm_size \times width/height} \right|$.
Ceci permettra de trouver le meilleur couple qui suit la longueur et la largeur : plus la largeur est grande, plus il y aura de nombre de processus en y, et inversement. Cela nous permettra d'avoir une topologie en halo et non en chaîne dans la majorité des cas.

Nous en profiterons également pour revisiter la manière dont les cellules fantômes sont envoyés horizontalement. En effet, pour ces communications, les processus envoient une colonne de cellules qui ne peuvent se faire un 1 seul message, contrairement aux envois verticaux, les colonnes n'étant pas contiguës en mémoire en C++. Heureusement, il est possible de le faire en 1 message par le biais de vecteur MPI [1]. Cela nous permet de faire un Datatype MPI permettant de faire un vecteur composé de *mesh->height - 2* quantité de *DIRECTIONS* doubles, avec un stride de *mesh->width * DIRECTIONS*. Ainsi, il suffit d'envoyer 1 vecteur de ce type par message pour envoyer une colonne en entier.

3.3. Implémentation OpenMP

La fonction *main* comporte une boucle for sur le nombre d'itérations à réaliser pour la simulation (dans notre cas,

1. Source : https ://www.to.infn.it/ mignone/MPI/lecture4.pdf

20,000 itérations) qui fait appel à plusieurs fonctions qui sont parallélisables dans un contexte multi-thread. C'est pourquoi nous avons englober cette partie dans une région parallèle OpenMP, puis nous avons réparti la charge de travail des threads à l'intérieur de ces fonctions parallélisables. Pour celles qui ne le sont pas nous avons rajoutons un "single".

Parmis les fonctions parallélisables nous retrouvons *special_cells*, *collision* et *propagation* qui font appel des boucles for sur la height et la width. Puisque par la division des taches entre processus MPI, nous avons réparti la height entre processus, alors nous allons répartir les taches sur la width entre les threads. Nous posons pour ces trois fonctions une parallélisation de la seconde boucle for sous un ordonnancement dynamique (déterminé expérimentalement avec plusieurs tailles de blocs comme étant la meilleure solution sur des coeurs à 2 coeurs logiques, répartissant MPI sur les coeurs physiques et OpenMP sur les hyperthreads).

Notons que nous devons laisser la barrière implicite à la fin de la boucle parallélisée afin d'assurer que les résultats restent justes.

3.4. Optimisation mémoire

Nous avons réaliser une première mesure du ratio $\frac{cache-misses}{cache-references}$ de chacune des fonctions du programme grace à perf, deux fonctions montraient un fort pourcentage de cache-misses, montrant une mauvaise utilisation de la mémoire. Ces fonctions sont (cf. Annexe D) :

- *collision* qui appelle *compute_cell_collision* et a 47.29% de cache-miss rate
- *propagation* a 26.70% de cache-miss rate

En analysant ces fonctions, nous voyons que nous gérons les tableaux Mesh : :cell de taille $width \times height \times DIRECTIONS = 802 \times (\frac{162}{\text{mpi_size}}) \times 9$. Leur type est des doubles, ce qui rend ces tableaux grands, il faut donc s'assurer de profiter de la localité temporelle des données afin d'éviter d'évincer la mémoire à cause d'autres variables ou calculs trop rapidement. En regardant la fonction *Mesh_get_cell* qui défini le "first-touch" sur ces tableaux, nous voyons que l'ordonnancement des tableaux est [height][width][DIRECTIONS], c'est pourquoi nous avons commencer par structurer l'ensemble des accès aux données dans tout le code comme un triple boucle for dans ce sens là. C'est-à-dire interchanger les boucles en width et height dans l'ensemble des fonctions ayant cette boucle for.

Commençons par la fonction *collision* :

Nous avons décider d'éviter de réaliser des appels de fonctions. En effet, *get_cell_velocity* et *compute_equilibrium_profile* ne sont pas inline car elles sont également appelées dans "initialization.cpp" et sont facilement adaptées pour une implémentation directe. Cependant, nous définissons *compute_cell_collision* comme étant inline. Nous initialisons la plupart des variables en dehors de la triple boucle for afin de réutiliser leurs localité mémoire, puis nous allégeons les calculs en terme d'opérations, par exemple de "f_eq".

Ensuite pour la fonction *propagation* :

Avant d'optimiser cette fonction, dans la version de base, la comparaison des résulats avec la référence était fausse, et avions besoin de corriger le fait que ce soti

Deux principaux problèmes à optimiser ont été recontrés ici : le premier est en relation avec les types des variables. Nous avons i,j,k des size_t ; ii,jj des ssize_t ; direction_matrix[k][] des doubles ; mesh_out->width, mesh_out->height des uint32_t ; *Mesh_get_cell* qui prend en argument des int. Les parenthèses sur la lignes ssize_t ii = (i + direction_matrix[k][0]) ; assure un premier cast de i en double pour une somme en double, puis un cast en ssize_t pour obtenir le résultat. Tout comme la fonction précédente, nous avons initiliaser au début de l'appel de la fonction les différentes variables et avons tout définis en tant que int pour éviter le plus de conversions possibles.

Le deuxième problème est la condition dans la triple boucle for, en analysant l'utilisation de cette fonction, nous voyons que c'est l'avant-dernière fonction de la boucle for dans *main* avant *save_frame_all_domain*. Or cette dernière n'opère pas sur les cellules fantômes, nous retirons donc des opérations sur les boucles en j et i ; cela vient également éviter d'avoir des valeurs de j et i très grandes lorsque i=0 et direction_matrix[k][0]=-1.0 par exemple.

Suite à ces optimisations, nous avons réévaluer dans les mêmes conditions les mesures de perf pour le cache-miss ratio (cf. Annexe D) :

- *collision* a 5.00% de cache-miss rate
- *propagation* a 5.72% de cache-miss rate

Nous notons donc une grande différence, non seulement dans le ratio, mais également dans le nombre de références éffectuées : 70 milliards contre 37 milliards, montrant bien une meilleure localité temporelle de la mémoire sur les parties les plus importantes du programme.

3.5. Autres optimisations

3.5.1. MPI_Syncall

Une optimisation notable réalisée est en relation avec la fonction *MPI_Syncall* appelée à la fin de la fonction établissant les communications. Or cette fonction ne fait pas partie de la librairie MPI.

Dans le code, tout ce qui est en relation avec MPI_Syncall se trouve ici :

```
MPI_Syncall(MPI_COMM_WORLD);

static MPI_Syncfunc_t* MPI_Syncall = PMPI_Syncall_cb;

static int PMPI_Syncall_cb(MPI_Comm comm) {
  static int (*__builtin_fence_ps)() = rt_tpl_sync(comm,
       __builtin_fence_ps, MPI_HINT_VTBL);
  return __builtin_fence_ps();
}

#define rt_tpl_sync(comm,fence,sym)
     catof(resolve,_tpl,_sync)<decltype(fence)>(sym)

#define catof(a,b,c) a##b##c

#define MPI_HINT_VTBL
     "5f5f6275696c74696e5f73796e635f66656e63655f"
```

Nous essayons donc de retracer ce que cela veut dire, premierement, nous interpretons que :

```
PMPI_Syncall_cb(MPI_COMM_WORLD) {
    __builtin_fence -> pointeur de fonction (?) qui
         devrait renvoyer un int
    return __builtin_fence; -> on appel la fonction
}
```

ce qui veut dire

```
return catof(resolve,_tpl,_sync)
    <__builtin_fence_ps>(MPI_HINT_VTBL)
```

Puisque ___builtin_fence_ps est utilisé comme un type dans les <>, nous extrayons que c'est un int(*)(), nous avons alors :

```
return resolve##_tpl##_sync<int(*)()>(
    "5f5f6275696c74696e"
    "5f73796e635f66656e"
    "63655f");
```

les hashtag servent a concaténer les arguments, donc :

```
return resolve_tpl_sync<int(*)()>(
    "5f5f6275696c74696e"
    "5f73796e635f66656e"
    "63655f"
);
```

on retrouve la fonction resolve_tpl_sync dans le fichier build/include/lbm/tpl_loader.hpp :

```
template <typename F>
F resolve_tpl_sync(const std::string& tbl_addr) {
 void* sym =
      dlsym(RTLD_DEFAULT,hex_decode(tbl_addr).data());
 return reinterpret_cast<F>(sym);
}
```

avec :

```
extern void *dlsym (void *__restrict __handle,
                  const char *__restrict __name) __THROW
                       __nonnull ((2));

#define RTLD_DEFAULT  ((void *) 0)
```

en remplaçant les arguments

```
dlsym(RTLD_DEFAULT, hex_decode(
    "5f5f6275696c74696e"
    "5f73796e635f66656e"
    "63655f"
).data());
```

En apprenant par coeur les hex codes, il est trivial que 5f5f6275696c74696e5f73796e635f66656e63655f = ___builtin_sync_fence_ Donc : dlsym(((void *) 0), "___builtin_sync_fence_") ;

Il parait que cette fonction retourne l'adresse d'un symbole de nom "___builtin_sync_fence_" dans toutes les librairies SHARED. Cela comprend en particulier top.lbm-lib, MPI ou OpenMP.

Nous appliquons ensuite reinterpret_cast<int(*)()>(sym) ce qui rend cette adresse une fonction pointée que nous pouvons appeler, car nous faisons return de ce pointeur.

La fonction de nom "___builtin_sync_fence_" se trouve dans build/include/lbm/tpl.hpp et est définie comme un code en assembleur qui implique une veille de une seconde, sans raison particuliere.

Pour la retirer, puisque nous avons vu le contenu des deux fichiers "tpl_loader.hpp" et "tpl.hpp" alors nous allons effacer la ligne

```
FILES ${CMAKE_BINARY_DIR}/include/lbm/tpl.hpp
     ${CMAKE_BINARY_DIR}/include/lbm/tpl_loader.hpp
```

du CMakeFile dans src/lbm, et effacer également toutes les lignes en relation avec MPI_Syncall.

3.5.2. Autres

Nous avons retirer la fonction *init_cond_velocity_0_density_1* qui n'est pas utilisée.

En termes de linking : nous avons limiter les includes en fonction des dépendances des fichiers, et nous sommes assurer de ne pas inclure plusieurs fois les mêmes fichiers, par exemple inclure une fois dans un .hpp puis le même fichier dans le .cpp associé.

Les boucles for sur DIMENSIONS ont été déroulées car DIMENSIONS=2.

4. Expériences

4.1. Protocole expérimental

Notre approche pour vérifier les optimisations que nous ferons est la suivante : pour chaque optimisation, nous mesurerons la variation de fréquence de mise à jour des cellules (en MLUPS) et vérifierons que les résultats restent vrais en les comparant avec une référence.

Toutes les mesures ont été fait sur la configuration système suivante :
Chip : Apple M4
Total Number of Cores : 10 (4 Performance and 6

Efficiency)
Memory : 16 GB
Page Size : 16 KB

Cores Performance :
Frequency : 4,4 GHz
L1i : 192 KB
L1d : 128 KB
L2 : 16 MB

Cores Efficiency :
Frequency : 2,9 GHz
L1i : 128 KB
L1d : 64 KB
L2 : 4 MB

Compiler :
g++ : 15.0.2

4.1.1. Mesure du nombre d'opérations par secondes

Le programme intègre déjà une mesure en MLUPS (Million Lattice Updates Per Second), déterminée comme suit :

$$Mlups = \frac{total_cells \cdot ITERATIONS}{elapsed_time}$$

avec

- $total_cells$: le nombre total de cellules dans le maillage
- $ITERATIONS$: le nombre total de mises à jour faites par le programme
- $elapsed_time$: le runtime de la boucle de calcul

Nous mesurerons cette quantité 3 fois, puis nous analyserons la différence avec la précédente :

$$Mlups_{final} = \sum_{i=1}^{3} \frac{\left|Mlups^{precedent} - Mlups_i^{mesure}\right|}{3}$$

Les incertitudes seront déterminées par la déviations standard de nos valeurs mesurées.

4.2. Vérification des résultats

Afin de d'assurer la cohérence de notre modèle suite aux optimisations apportées, nous quantifierons celles-ci en analysant l'écart entre les données de référence (cf. Table 1) et celles mesurées.
Pour ce faire, nous utiliserons le script *Python* fourni qui consiste à sommer toutes les valeurs de densité ρ et la norme du vecteur vitesse $U(\mathbf{x}, t)$ de toutes les cellules pour chaque frame calculée du jeu de donnée de référence ainsi que pour chaque frame après optimisation, avant de comparer les résultats. La solution étant déterministe, les résultats devront être identiques.

Table 1 – Configuration de référence

Paramètres	**Valeurs**
Taille de la conduite	800×160
Position de l'obstacle	$\{100, 80\}$
Rayon de l'obstacle	0.16
Nombre de Reynolds de l'écoulement Re	96
Vitesse maximal du fluide $U(x = 0, y, t)$	0.16

Ainsi, nous lancerons les commandes suivantes :

```
uv venv
uv run lbm-viz --check\
 ref_results.raw results.raw
```

Les mesures seront fait avec les flags d'optimisations *-O3* et *-march=native*. Le compilateur utilisé sera indiqué pour chaque série de mesures.

5. Résultats

Voici un tableau récapitulatif des optimisations et leurs impacts sur le Mlups (cf Table 2), la valeur Mlups mesurée pour la version initale est $Mlups = 125 \pm 3$:

Table 2 – Optimisation de la communication dans l'halo

Optimisation	**Mlups**	δ_{Mlups}
Comm. non-bloquantes	1575	± 20
Sup. *MPI_Barrier* main	2363	± 20
Halo + 1 message / colonne	2503	± 30

Table 3 – Optimisation memoire

Optimisation	**Mlups**	δ_{Mlups}
Optimisation memoire	1830	± 30

Discussions :
Nous remarquons que les améliorations faites sur les communications ont des conséquences plus importantes sur le programme. Cependant, on constate qu'avec les optimisations mémoires, les performances diminuent.

La scalabilité de notre programme (cf. Annexe E) montre qu'il y a encore beaucoup d'optimisations à faire pour avoir un programme bien optimisé.

6. Conclusion

Au long de l'analyse de ce programme, nous avons été capables de repérer divers types de problèmes optimisables. Que ce soit en lien avec le modèle de parallélisation, avec des phénomènes informatiques ou encore des observations directes de mesures de performances. Ces problèmes nous

ont mener par la suite à une inspection fine du code et de compréhension du phénomène étudié.

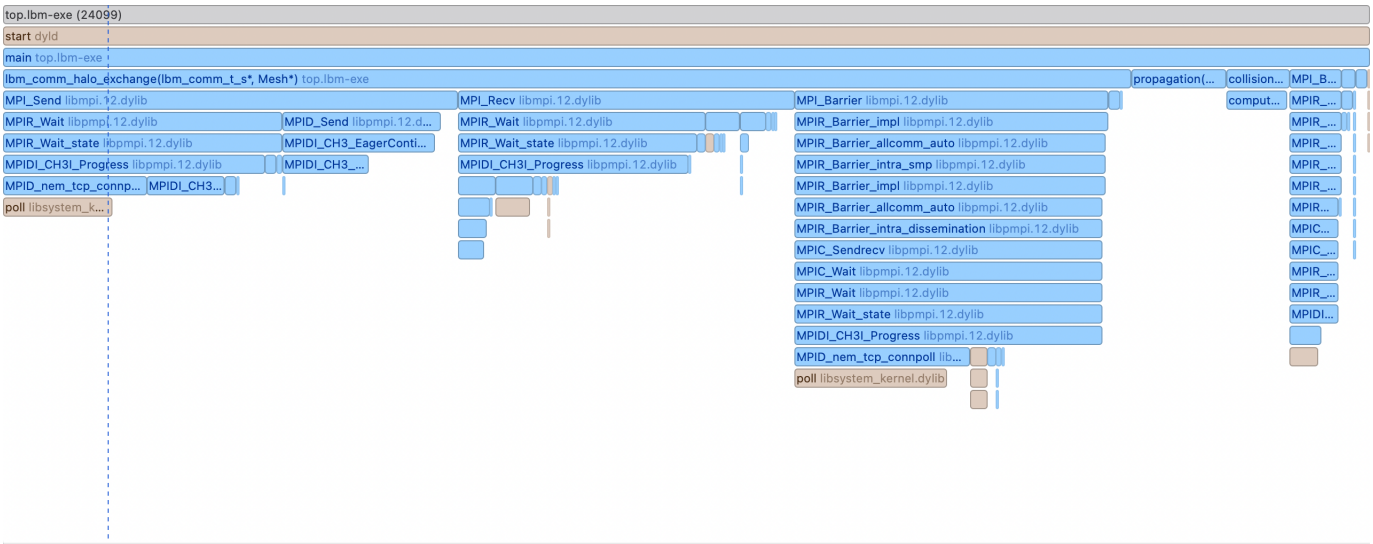
Le résultat obtenu à ce jour est satisfaisant, la différence lors des comparaisons suite à chaque optimisations est notable et nous avons étudier une partie importante de l'ensemble du code.

Parmis les optimisations supplémentaires qui seraient réalisables nous comptons principalement la vectorisation. Les fonctions *collision* et *propagation* sont complètement vectorisables (après optimisation) mais nécessiteraient une approche différente de l'ordonnancement de la mémoire des tableaux des cellules, en particulier d'avoir la width contigüe en mémoire.

Finalement ce projet à été enrichissant et nous a obligé à appliquer concrètement diverses connaissances apprises au cours de cette année d'étude.

Annexes

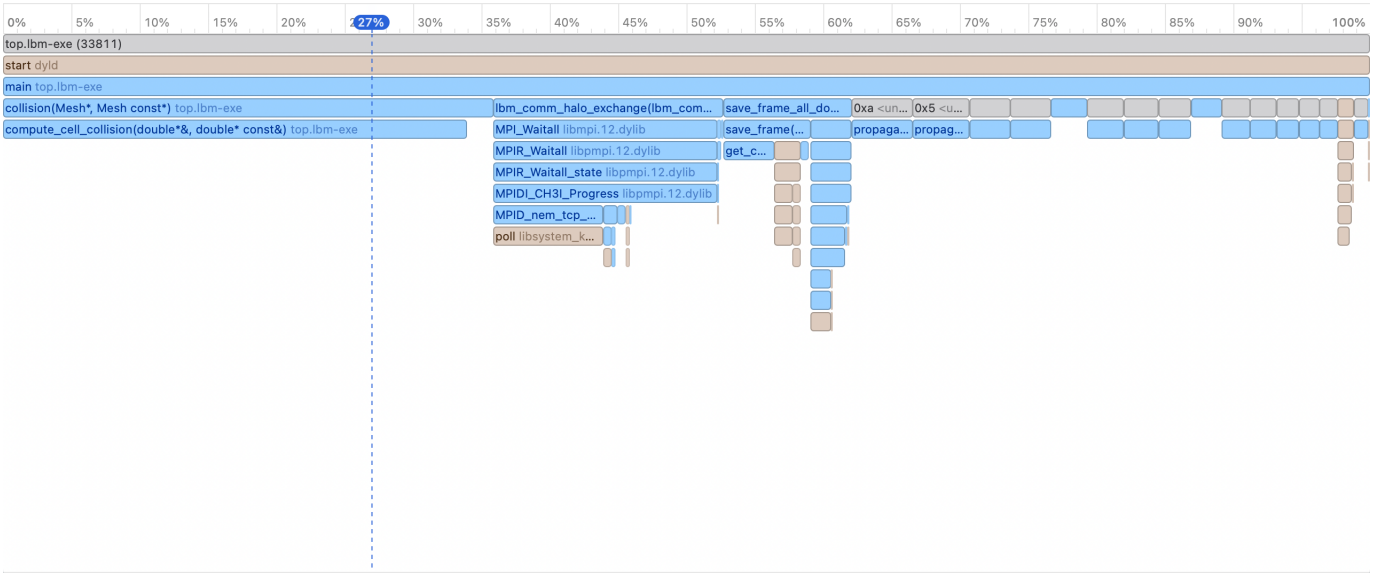
A. Flame Graph initial : Instruments.app



B. Fonction *lbm_comm_halo_exchange* initiale

```
void lbm_comm_halo_exchange(lbm_comm_t* mesh, Mesh* mesh_to_process) {
  int rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);

  lbm_comm_sync_ghosts_horizontal(mesh, mesh_to_process, COMM_SEND, mesh->right_id, mesh->width - 2);
  lbm_comm_sync_ghosts_horizontal(mesh, mesh_to_process, COMM_RECV, mesh->left_id, 0);
  MPI_Barrier(MPI_COMM_WORLD);

  lbm_comm_sync_ghosts_horizontal(mesh, mesh_to_process, COMM_SEND, mesh->left_id, 1);
  lbm_comm_sync_ghosts_horizontal(mesh, mesh_to_process, COMM_RECV, mesh->right_id, mesh->width - 1);
  MPI_Barrier(MPI_COMM_WORLD);

  lbm_comm_sync_ghosts_vertical(mesh_to_process, COMM_SEND, mesh->bottom_id, mesh->height - 2);
  lbm_comm_sync_ghosts_vertical(mesh_to_process, COMM_RECV, mesh->top_id, 0);
  MPI_Barrier(MPI_COMM_WORLD);

  lbm_comm_sync_ghosts_vertical(mesh_to_process, COMM_SEND, mesh->top_id, 1);
  lbm_comm_sync_ghosts_vertical(mesh_to_process, COMM_RECV, mesh->bottom_id, mesh->height - 1);
  MPI_Barrier(MPI_COMM_WORLD);

  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_SEND, mesh->corner_id[CORNER_TOP_LEFT], 1, 1);
  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_RECV, mesh->corner_id[CORNER_BOTTOM_RIGHT], mesh->width - 1, mesh->height - 1);
  MPI_Barrier(MPI_COMM_WORLD);

  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_SEND, mesh->corner_id[CORNER_BOTTOM_LEFT], 1, mesh->height - 2);
  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_RECV, mesh->corner_id[CORNER_TOP_RIGHT], mesh->width - 1, 0);
  MPI_Barrier(MPI_COMM_WORLD);

  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_SEND, mesh->corner_id[CORNER_TOP_RIGHT], mesh->width - 2, 1);
  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_RECV, mesh->corner_id[CORNER_BOTTOM_LEFT], 0, mesh->height - 1);
  MPI_Barrier(MPI_COMM_WORLD);

  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_SEND, mesh->corner_id[CORNER_BOTTOM_LEFT], 1, mesh->height - 2);
  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_RECV, mesh->corner_id[CORNER_TOP_RIGHT], mesh->width - 1, 0);
  MPI_Barrier(MPI_COMM_WORLD);

  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_SEND, mesh->corner_id[CORNER_BOTTOM_RIGHT], mesh->width - 2, mesh->height - 2);
  lbm_comm_sync_ghosts_diagonal(mesh_to_process, COMM_RECV, mesh->corner_id[CORNER_TOP_LEFT], 0, 0);
  MPI_Barrier(MPI_COMM_WORLD);

  lbm_comm_sync_ghosts_horizontal(mesh, mesh_to_process, COMM_SEND, mesh->left_id, 1);
  lbm_comm_sync_ghosts_horizontal(mesh, mesh_to_process, COMM_RECV, mesh->right_id, mesh->width - 1);

  MPI_Syncall(MPI_COMM_WORLD);
}
```

C. Flame Graph après optimisation MPI : Instruments.app



D. Comparaison version naïve et optimisée de cache-miss ratio avec perf

```
Samples: 4M of event 'cache-misses', Event count (approx.): 27034120635
  Children      Self  Command      Shared Object                Symbol
+   22.36%    22.08%  top.lbm-exe  libtop.lbm-lib.so            [.] propagation(Mesh*, Mesh const*)
+   21.03%     0.00%  top.lbm-exe  [unknown]                    [.] 0x0000000000000001
+   16.10%     0.00%  top.lbm-exe  [unknown]                    [k] 0x30ec834853555441
+   16.10%     0.00%  top.lbm-exe  libmpi.so.40.40.7            [.] mca_pml_ob1_del_procs
+   13.86%    13.86%  top.lbm-exe  libmpi.so.40.40.7            [.] 0x0000000000286dee
+   11.89%    11.78%  top.lbm-exe  libmpi.so.40.40.7            [.] mca_pml_ob1_recv_req_start
+   11.68%     0.00%  top.lbm-exe  [unknown]                    [.] 0x766365725f31626f
     9.57%     9.41%  top.lbm-exe  libtop.lbm-lib.so            [.] compute_cell_collision(double*, double*
     8.46%     8.32%  top.lbm-exe  libtop.lbm-lib.so            [.] compute_equilibrium_profile(double*, double, int)
+    4.38%     0.00%  top.lbm-exe  [unknown]                    [k] 0000000000000000
+    4.33%     0.00%  top.lbm-exe  [unknown]                    [.] 0x6d6d6f635f31626f
+    4.08%     4.03%  top.lbm-exe  libmpi.so.40.40.7            [.] mca_pml_ob1_recv
+    3.32%     0.00%  top.lbm-exe  [unknown]                    [.] 0x00745f6570797461
+    3.09%     0.00%  top.lbm-exe  [unknown]                    [.] 0x0000000000000016
+    3.04%     0.00%  top.lbm-exe  libmpi.so.40.40.7            [.] 0x00007f3e95486dee
+    2.97%     0.00%  top.lbm-exe  [unknown]                    [.] 0x00007f3e95578ce0
```

(a) Cache-misses version naïve

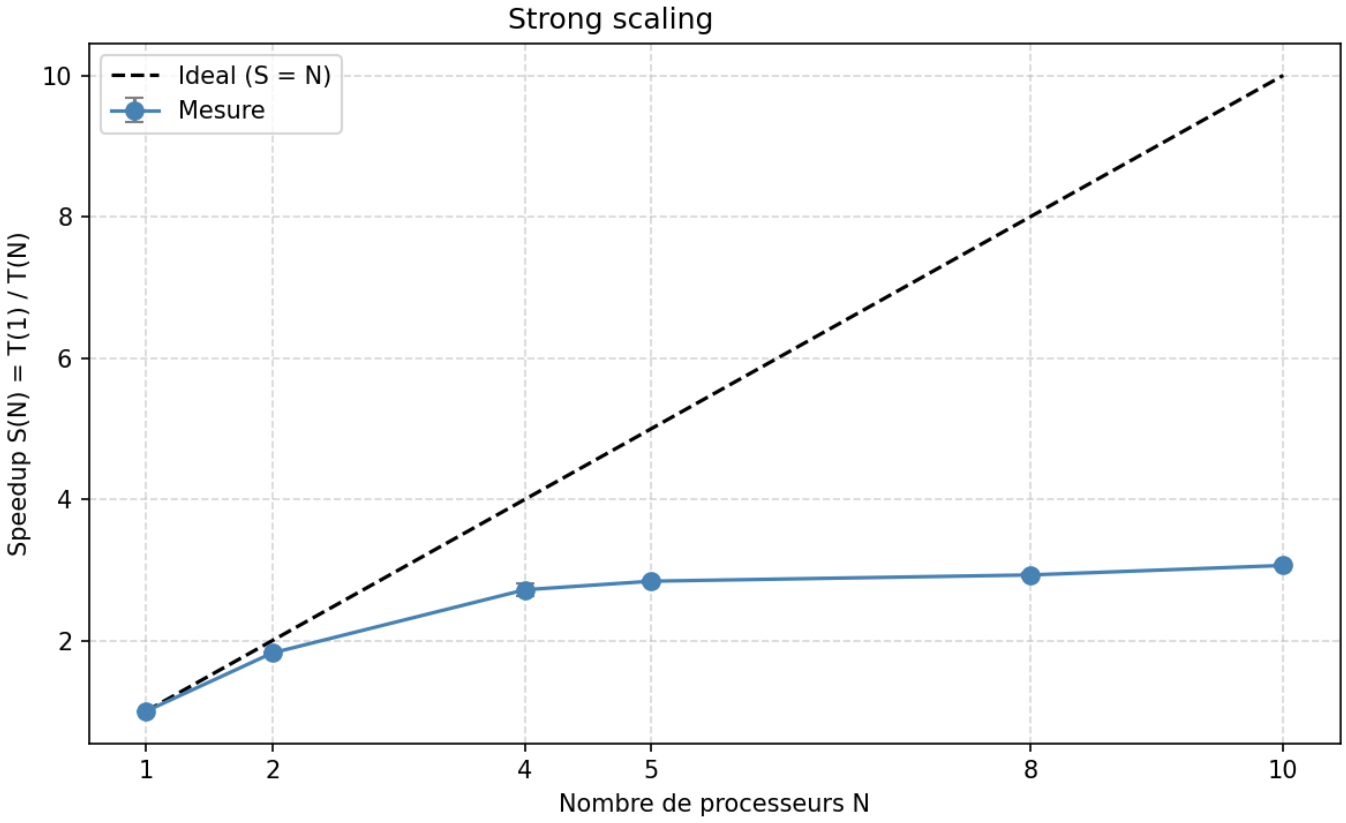
```
Samples: 4M of event 'cache-references', Event count (approx.): 70679141601
  Children      Self  Command      Shared Object                Symbol
-   31.66%    31.51%  top.lbm-exe  libtop.lbm-lib.so            [.] propagation(Mesh*, Mesh const*)
     31.65% propagation(Mesh*, Mesh const*)
+   18.62%     0.00%  top.lbm-exe  [unknown]                    [.] 0x0000000000000001
+   10.73%     0.00%  top.lbm-exe  [unknown]                    [.] 0x30ec834853555441
+   10.73%     0.00%  top.lbm-exe  libmpi.so.40.40.7            [.] mca_pml_ob1_del_procs
+    9.25%     9.19%  top.lbm-exe  libmpi.so.40.40.7            [.] mca_pml_ob1_recv_req_start
+    9.05%     0.00%  top.lbm-exe  [unknown]                    [.] 0x766365725f31626f
+    8.95%     8.95%  top.lbm-exe  libmpi.so.40.40.7            [.] 0x0000000000286dee
     7.69%     7.61%  top.lbm-exe  libtop.lbm-lib.so            [.] compute_cell_collision(double*, double*)
     7.23%     7.16%  top.lbm-exe  libtop.lbm-lib.so            [.] compute_equilibrium_profile(double*, double, int)
+    3.92%     0.00%  top.lbm-exe  [unknown]                    [k] 0000000000000000
+    3.67%     0.00%  top.lbm-exe  [unknown]                    [.] 0x00745f6570797461
+    3.60%     3.58%  top.lbm-exe  libmpi.so.40.40.7            [.] mca_pml_ob1_recv
+    3.10%     0.00%  top.lbm-exe  [unknown]                    [.] 0x00007febbf578ce0
+    2.98%     0.00%  top.lbm-exe  [unknown]                    [.] 0x6d6d6f635f31626f
+    2.45%     0.00%  top.lbm-exe  [unknown]                    [.] 0x0000000000000016
```

(b) Cache-references version naïve

```
Samples: 1M of event 'cache-misses', Event count (approx.): 2760605927
  Children      Self  Command      Shared Object                Symbol
+   36.12%    34.71%  top.lbm-exe  top.lbm-exe                  [.] propagation(Mesh*, Mesh const*)
+   21.95%    20.54%  top.lbm-exe  top.lbm-exe                  [.] collision(Mesh*, Mesh const*)
+   15.39%     0.22%  top.lbm-exe  [kernel.kallsyms]            [k] entry_SYSCALL_64_after_hwframe
+   15.11%     0.35%  top.lbm-exe  [kernel.kallsyms]            [k] do_syscall_64
+   12.38%     0.01%  top.lbm-exe  libc.so.6                    [.] process_vm_readv
+   12.31%     0.01%  top.lbm-exe  [kernel.kallsyms]            [k] __x64_sys_process_vm_readv
+   12.30%     0.06%  top.lbm-exe  [kernel.kallsyms]            [k] process_vm_rw
+   12.15%     0.10%  top.lbm-exe  [kernel.kallsyms]            [k] process_vm_rw_core.constprop.0
+   11.52%     0.00%  top.lbm-exe  [unknown]                    [k] 0x000000000000e100
+   11.02%    11.02%  top.lbm-exe  libgomp.so.1.0.0             [.] 0x0000000000012282
+    8.16%     0.06%  top.lbm-exe  [kernel.kallsyms]            [k] copy_page_to_iter
+    8.10%     0.17%  top.lbm-exe  [kernel.kallsyms]            [k] _copy_to_iter
     7.95%     7.82%  top.lbm-exe  [kernel.kallsyms]            [k] rep_movs_alternative
+    4.24%     0.03%  top.lbm-exe  [kernel.kallsyms]            [k] asm_sysvec_apic_timer_interrupt
+    4.13%     0.01%  top.lbm-exe  [kernel.kallsyms]            [k] sysvec_apic_timer_interrupt
+    4.08%     0.00%  top.lbm-exe  [unknown]                    [k] 0000000000000001
```

(c) cache-misses version optimisée

```
Samples: 1M of event 'cache-references', Event count (approx.): 37161024392
  Children      Self  Command      Shared Object                Symbol
+   45.25%    45.07%  top.lbm-exe  top.lbm-exe                  [.] propagation(Mesh*, Mesh const*)
    30.69%    30.51%  top.lbm-exe  top.lbm-exe                  [.] collision(Mesh*, Mesh const*)
+   12.83%    12.83%  top.lbm-exe  libgomp.so.1.0.0             [.] 0x0000000000012282
+    5.22%     0.08%  top.lbm-exe  [kernel.kallsyms]            [k] entry_SYSCALL_64_after_hwframe
+    5.12%     0.13%  top.lbm-exe  [kernel.kallsyms]            [k] do_syscall_64
+    4.18%     0.00%  top.lbm-exe  libc.so.6                    [.] process_vm_readv
+    4.17%     0.00%  top.lbm-exe  [kernel.kallsyms]            [k] __x64_sys_process_vm_readv
+    4.16%     0.01%  top.lbm-exe  [kernel.kallsyms]            [k] process_vm_rw
+    4.14%     0.02%  top.lbm-exe  [kernel.kallsyms]            [k] process_vm_rw_core.constprop.0
+    3.60%     0.00%  top.lbm-exe  [unknown]                    [k] 0x000000000000e100
+    3.56%     0.02%  top.lbm-exe  [kernel.kallsyms]            [k] copy_page_to_iter
+    3.55%     0.07%  top.lbm-exe  [kernel.kallsyms]            [k] _copy_to_iter
     3.50%     3.48%  top.lbm-exe  [kernel.kallsyms]            [k] rep_movs_alternative
+    1.83%     0.00%  top.lbm-exe  [unknown]                    [k] 0000000000000001
+    1.67%     0.00%  top.lbm-exe  libgomp.so.1.0.0             [.] 0x00007f47530b3282
+    1.67%     0.00%  top.lbm-exe  libgomp.so.1.0.0             [.] 0x00007f085ebf6282
```

(d) cache-references version optimisée

E. Etude de scalabilité du programme optimisé